

SADE — Safe Media Delivery

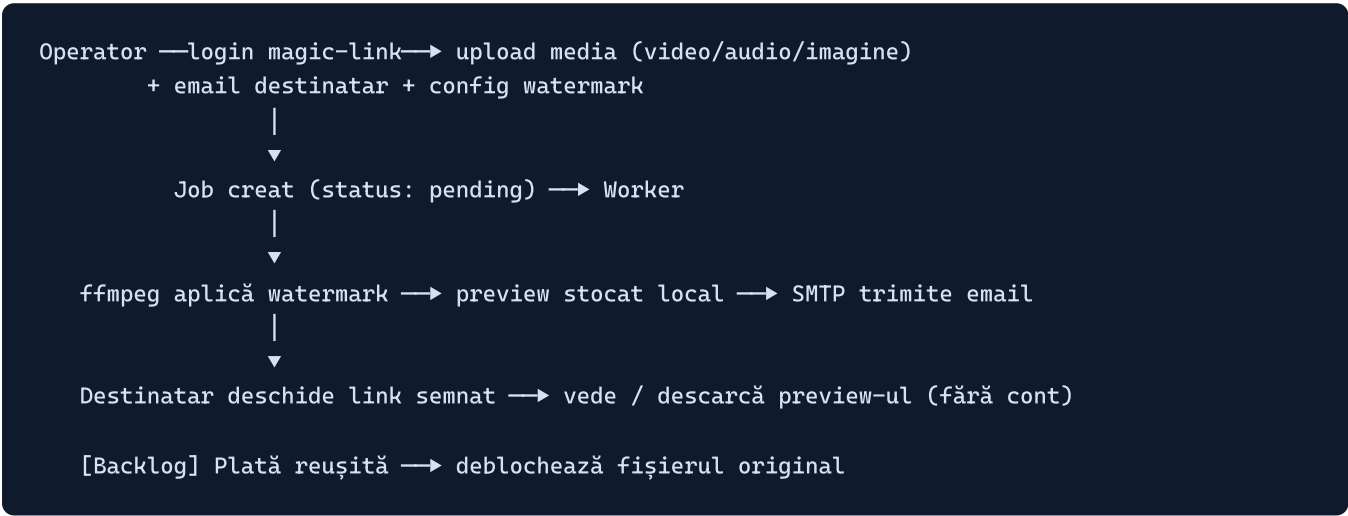
Plan de arhitectură și implementare

Document de lucru · 2 septembrie 2026 · structură aliniată la `nutrition-planner`

1. Obiectiv

SADE aplică automat un **watermark** pe o sursă media (video / audio / imagine), apoi notifică destinatarul prin **email** cu un link către preview-ul watermark-uit.

Backlog: plăți — după o tranzacție reușită, destinatarul primește acces la fișierul original.



2. Stack tehnologic

Zonă	Alegere	Note
Backend	Go 1.26, <code>net/http</code> cu routing stdlib (1.22+)	fără framework greu
Bază de date	PostgreSQL + driver <code>pgx/v5/stdlib</code>	<code>sql.Open("pgx", ...)</code> ; Docker Compose în dev
Strat de date	<code>schema-builder</code> (DDL la boot) + <code>crud-depot</code> (CRUD runtime)	structuri model comune, dialect <code>Postgres{}</code>
Media	<code>u2takey/ffmpeg-go</code>	necesită binarul <code>ffmpeg</code> / <code>ffprobe</code> pe PATH
Notificări	<code>internals/mailer</code> — SMTP (<code>net/smtp</code>)	înlocuiește <code>notifix</code> (produs in-house, inaccesibil)
Logging	<code>internals/logger</code> — <code>log/slog</code> + <code>lumberjack</code>	înlocuiește <code>log-dog</code> ; pattern identic cu nutrition-planner
Auth	magic-link custom + sesiune cookie DB-backed	revocare ușoară
Frontend	SvelteKit 2 / Svelte 5 (runes, TS)	<code>adapter-static</code> , servit de Go
Storage	interfața <code>Storage</code> ; impl. <code>local</code> acum, <code>s3</code> ulterior	local pentru început, S3 în producție

3. Convenția pe feature

Fiecare pachet din `internals/app/<feature>/` are aceleași fișiere (model identic cu `nutrition-planner`):

Fișier	Rol
<code>model.go</code>	struct domeniu cu tag-uri <code>db:</code> (schema-builder + crud-depot) + <code>TableName()</code> ; DTO-uri <code>CreateRequest</code> / <code>UpdateRequest</code> / <code>Response</code> cu tag-uri <code>json:</code> ; <code>toResponse()</code> . Domeniul nu ajunge niciodată în UI.
<code>errors.go</code>	<code>ErrNotFound</code> , <code>ErrInvalidInput</code> , ... la nivel de pachet
<code>repo.go</code>	interfața <code>Repo</code> + <code>repo{ depot *crud.CRUD }</code> , <code>NewRepo(db *database.Database, deps...)</code> , mapează <code>crud.ErrNotFound</code> → <code>ErrNotFound</code>
<code>service.go</code>	interfața <code>Service</code> + <code>service{ repo, log *slog.Logger }</code> , validare, log pe Create/Update/Delete (nu pe read)
<code>handler.go</code>	<code>Handler{ svc Service }</code> , <code>NewHandler(svc)</code> . Pentru SADE: metodele sunt <code>http.HandlerFunc</code> (decode request → svc → <code>writeJSON</code>), înregistrate în <code>internals/app/router.go</code>
<code>*_test.go</code>	<code>repo_test.go</code> , <code>service_test.go</code>

4. Arborele proiectului

```
sade/
├── main.go           # subțire: NewApp() → server.Start → shutdown grațios
├── app.go            # NewApp(): tot wiring-ul (ca în nutrition-planner)
├── config.yaml / .env # auto-generate la prima rulare
├── docker-compose.yml # Postgres pentru dev
├── go.mod
├── cmd/
│   └── seed/main.go  # utilitar opțional (user demo)
├── config/
│   ├── struct.go     # Config: Server, Database, Logger, Storage,
│   │                 # Mailer, Auth, FFmpeg, Worker, Upload
│   ├── main.go       # Load(), generateConfig(), loadEnvOverrides,
│   │                 # validateSecrets, MaskSecrets
│   └── *_test.go
├── internals/
│   ├── app/
│   │   ├── server.go # http.Server lifecycle (Start / Shutdown)
│   │   ├── router.go # mux: montează rutele fiecărui Handler
│   │   ├── middleware.go # RequestLog, Recover, SessionAuth, RateLimit, CORS
│   │   ├── httpx.go    # writeJSON / decodeJSON / error→status
│   │   ├── user/       # model errors repo service handler (+tests)
│   │   ├── auth/       # MagicToken + Session; request / callback / logout / me
│   │   ├── job/        # upload multipart + list + detail + status
│   │   ├── share/      # /p/:token, /d/:token - token semnat HMAC, fără login
│   │   └── payment/    # BACKLOG
│   ├── database/      # struct.go + main.go (driver=postgres)
│   ├── logger/        # New(cfg) (*slog.Logger, io.Closer, error)
│   ├── storage/       # storage.go (interfața) + local.go [s3.go mai târziu]
│   ├── ffmpeg/        # engine.go probe.go video.go audio.go image.go (+tests)
│   ├── worker/        # worker.go - pool care preia job-urile pending
│   ├── mailer/        # mailer.go (SMTP) + templates/ (magic-link, preview-ready)
│   ├── token/         # sign.go / verify.go - HMAC pentru share tokens
│   └── testutil/      # database.go - Postgres de test
└── frontend/          # SvelteKit; src/lib: api.ts stores.ts types.ts il8n.ts theme.ts
```

5. Diferența față de nutrition-planner

nutrition-planner este o aplicație Wails (handler = metode bind-uite la frontend). SADE este **HTTP server + web**, deci:

- **handler.go** per feature expune **http.HandlerFunc** -uri (decode request → **svc** → **writeJSON**), înregistrate central în **internals/app/router.go**.
- Apare stratul **internals/app/server.go** + **middleware.go**.
- Frontend-ul rămâne **SvelteKit** (nu Vite+Svelte simplu), vorbește cu API-ul prin **src/lib/api.ts**.

6. Model de date

Structuri cu tag-uri **db:**, în **model.go** -ul fiecărui feature; folosite atât de **schema-builder** cât și de **crud-depot**.

Model	Câmpuri cheie
User	<code>id uuid pk · email unique,index,notnull · role default:operator · created_at oncreate · updated_at onwrite</code>
MagicToken	<code>id uuid pk · user_id references users(id),on_delete:cascade · token_hash unique,index · expires_at · used_at (null) · created_at oncreate</code>
Session	<code>id uuid pk · user_id → users,cascade · token_hash unique,index · expires_at · created_at oncreate</code>
Job	<code>id uuid pk · user_id → users · status default:pending (pending/processing/done/failed) · media_type (video/audio/image) · recipient_email notnull · watermark_kind (logo/text/both) · watermark_text (null) · watermark_opts (jsonb) · original_asset_id → assets · preview_asset_id (null) · error (null) · created_at oncreate · updated_at onwrite</code>
Asset	<code>id uuid pk · job_id → jobs,cascade · kind (original/preview) · path notnull · filename · mime · size_bytes · checksum · created_at oncreate</code>
Payment (backlog)	<code>id uuid pk · job_id → jobs · provider · provider_ref · amount_cents · currency · status · created_at oncreate</code>

7. API HTTP

```

POST /api/auth/request      {email}                → trimite magic link
GET  /api/auth/callback?token=...  → setează sesiune, redirect în frontend
POST /api/auth/logout
GET  /api/me
POST /api/jobs              multipart: file + recipient_email + opts → creează Job + enqueue
GET  /api/jobs              lista job-urilor proprii
GET  /api/jobs/:id          detaliu + status
GET  /p/:token              pagină / stream preview pentru destinatar (token semnat, fără login)
GET  /d/:token              download preview watermark-uit
# backlog:
POST /api/jobs/:id/pay      checkout; webhook → deblocare; GET /d/original/:token

```

Middleware: logging cereri, recover panică, auth sesiune pe `/api/*` (mai puțin `auth`), rate-limit pe `/api/auth/request`.

8. Motor de watermark (ffmpeg-go)

- **Video** — filtru `overlay` pentru logo PNG (poziție / opacitate / scală din `watermark_opts`) și/sau `drawtext` pentru text dinamic (email destinatar + timestamp). Reencode H.264 / AAC, `+faststart`.
- **Audio** — mixează peste track un asset audio de watermark pre-înregistrat, la volum redus, repetat la interval configurabil (`amix` + `adelay` / `alooop`). TTS este în afara v1.
- **Imagine** — `overlay` / `drawtext` pe un singur cadru, păstrează formatul.

- Comun: `ffprobe` pentru validare → construire filtergraph → rulare → verificare output → înregistrare `Asset` preview.

9. Worker

Pool de goroutine in-process. DB = sursă de adevăr (`SELECT ... FOR UPDATE SKIP LOCKED` pe `status='pending'`), enqueue rapid din handler prin canal. Tranziții: `pending` → `processing` → `done/failed` , cu `error` la eșec și retry cu backoff (max N). La `done` : creează `Asset` preview + token semnat HMAC → trimite email.

10. Auth (magic link)

`request` → user după email (creat dacă nu există), token random 32B, se salvează `SHA-256(token)` + expirare 15 min, email cu `.../api/auth/callback?token=RAW` . `callback` → hash, caută token nefolosit & nevalidat, marchează `used_at` , creează `Session` → cookie `sade_session` (HttpOnly, Secure, SameSite=Lax). Token-uri publice pentru `/p` și `/d` : `HMAC(asset_id | exp, SHARE_SECRET)` — fără rând în DB.

11. Configurare (din env / yaml)

`Config` cu secțiuni: `Server` (Addr, PublicBaseURL, FrontendURL, timeouts), `Database` (driver=postgres, host, port, user, password `secret`, name, pooling), `Logger` (level, format, output, rotație), `Storage` (Driver=local, LocalDir; S3* ulterior), `Mailer` (Host, Port, User, Password `secret`, From, TLS), `Auth` (SessionSecret `secret`, ShareTokenSecret `secret`, MagicLinkTTL, SessionTTL), `FFmpeg` (BinPath, ProbePath, default-uri watermark), `Worker` (Concurrency, MaxRetries), `Upload` (MaxMB, AllowedMIME). `config.Load` auto-generează `config.yaml` + `.env` la prima rulare și validează secretele.

12. Frontend (SvelteKit)

Rute: `/` landing · `/login` (email → „verifică inbox-ul”) · `/app` (dashboard: formular upload + listă job-uri cu polling status) · `/app/jobs/[id]` · `/preview/[token]` (player + download pentru destinatar, fără auth). Client API în `src/lib/api.ts` , auth pe cookie. `adapter-static` (SPA), Go servește `frontend/build` .

13. Milestones

#	Livrabil
M0	<code>main.go</code> / <code>app.go</code> , <code>config</code> (struct + Load), <code>database</code> (postgres), <code>logger</code> , <code>schema.New</code> gol, <code>/healthz</code> , <code>docker-compose</code> Postgres
M1	pachetele <code>user/</code> + <code>auth/</code> , magic link, sesiune cookie DB-backed, <code>/api/me</code> ; <code>mailer</code> SMTP (dev = scrie emailul în log)
M2	<code>storage/local</code> , <code>job/</code> (<code>POST /api/jobs</code> multipart, list, detail), rânduri <code>Asset</code>
M3	<code>ffmpeg</code> engine: video overlay + drawtext (test cu clip sample) → audio → imagine
M4	<code>worker</code> pool + tranziții status, preview asset, <code>token/</code> HMAC, <code>share/</code> (<code>/p</code> , <code>/d</code>), email „preview gata”
M5	frontend: login, dashboard upload + polling status, pagină preview destinatar; <code>adapter-static</code> servit de Go
M6	rate-limit, validări, plafoane upload, retry, test integrare flux complet, README
Backlog	<code>payment/</code>

14. Decizii confirmate

1. **Entry point:** root `main.go` + `app.go` ca în nutrition-planner; se șterge `cmd/main.go` gol; `cmd/` rămâne pentru utilitare (seed).
2. **Driver Postgres:** `pgx/v5/stdlib` — `sql.Open("pgx", ...)`.
3. **Frontend:** rămâne SvelteKit.
4. **Notificări / logging:** SMTP (`internals/mailer`) + `slog` (`internals/logger`) — `notifix` și `log-dog` sunt produse in-house ale clientului, fără acces.

Următorul pas: implementarea M0.